const prvalues in the conditional operator

Document #:	P3177R0
Date:	2024-03-17
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

The conditional (or ternary) operator (7.6.16 [expr.cond]) is one of the most surprisingly complex parts of the language. The goal of this paper is to fix a surprising interaction in it, that is also causing an issue that we'd otherwise have to resolve in the library.

The issue at hand is about this:

```
// a version of declval but which doesn't add rvalue references
template <class T>
auto make() -> T;

template <class T, class U>
using cond = decltype(true ? make<T>() : make<U>());

template <class T>
using cref = cond<T, T const&>;
```

What do you expect the type of `cref<T>` to be? Well, it's obviously some kind of a `T` (since both sides are `T`, so nothing to convert). It cannot be `T&` (which cannot bind to either operand) or `T&&` (which cannot bind to `T const&`). It could potentially have been `T const&` (to do conditional lifetime extension) but it would be odd to elevate the prvalue to lvalue in this context. `T const&&` is a type that most people never think of, and I think in this context we should also not think about it.

I think at this point many people would expect that this leaves `T` amongst the options and conclude that `cref<T>` is simply `T`. But it turns out that's not quite the case. For *scalar types*, `cref<T>` is `T`. But for *class types*, `cref<T>` is actually `T const`.

This has some negative consequences, in addition to simply being a surprising difference.

§ 1.1 Pessimizing Assignment

First, consider this case:

```
auto get() -> T;
auto lookup() -> T const&;

T obj;
obj = flag ? get() : lookup();
```

For scalar types, this is a copy anyway, doesn't matter. But for class types, because the right-hand side is a `T const`, that means that this is always a copy-assignment. Or, put differently, this is the different behavior based on whether the conditional operator yielded `T const` or `T` in this case:

flag	`T const`	`T`
`true`	copy assignment	move assignment
`false`	copy construction then copy assignment	copy construction then move assignment

Note that either way this is less efficient than `if (flag) { obj = get(); } else { obj = lookup(); }` which is either a move assignment or a copy assignment without the extra potential copy construction, which is unfortunate.

This already seems like a relic of a pre-move-semantics era.

§ 1.2 Extra Wrapping

A second issue comes on up on the library side with ranges.

C++23 introduced a `views::as_const` [P2278R4] which is a range adaptor that tries to wrap a range, if necessary, to make it a constant range. It detects whether such wrapping is necessary by checking to see if the range's reference type would change by wrapping. That formula is, from 25.5.3.2 [const.iterators.alias]

```
template<indirectly_readable It>
  using iter_const_reference_t =
    common_reference_t<const iter_value_t<It>&&, iter_reference_t<It>>;

template<class It>
  concept constant-iterator =
        // exposition only
    input_iterator<It> && same_as<iter_const_reference_t<It>,
        iter_reference_t<It>>;

template<input_iterator I>
  using const_iterator = see below;
```

1 *Result*: If `I` models *constant-iterator*, `I`. Otherwise, `basic_const_iterator<I>`.

Now, consider an iterator over a non-proxy prvalue:

```
template <class T>
struct Priterator {
  using value_type = T:
  auto operator*() const -> T;

  // other stuff
};
```

While `common_reference` is the most complex type trait in the standard library, in this particular case it reduces to the much more manageable:

```
template <class T>
auto make() -> T;

template <class I>
using const_reference_t = cond<iter_value_t<I> const&&,
        iter_reference_t<I>>;
```

Which, for `Priterator<T>`, is `cond<T const&&, T>`.

This is the same construct we saw at the beginning of the paper, just that it's a `T const&&` instead of a `T const&`. But the rules end up being the same: if `T` is a scalar type, `const_reference_t<T>` is `T`. If `T` is a class type, then `const_reference_t<T>` is `T const`.

The result of this is that `Priterator<int>` *is* considered a constant iterator (because its `iter_const_reference_t` is `int`, which is the same as its reference type) while `Priterator<SomeClass>` *is not* considered a constant iterator (because its `iter_const_reference_t` becomes `SomeClass const`, which is now a different type). Which means that a range of prvalue class type isn't considered a constant range, and `views::as_const` would pointlessly wrap it.

This isn't just unnecessary wrapping and template instantiation - this now means instead of a range of prvalues, we end up with a range of *const* prvalues - which means copying when we could have potentially been moving.

At the very least, this is an issue we have to solve in the library (whether by special-casing this in the *`constant-iterator`* logic or, less desirably, in the `common_reference` logic). But this isn't uniquely a problem with `views::as_const`, it's simply the utility in which I first ran into this issue.

§ 2 Status Quo

The conditional operator between two operands of the same underlying type (excluding value category and const-ness) produces the following today for scalar vs class types:

?:	T	T const	T&	T const&	T&&	T const&&
T	T	T | T const	T	T | T const	T	T | T const

?:	T	T const	T&	T const&	T&&	T const&&
T const		T | T const	T | T const	T | T const	T | T const	T | T const
T&			T&	T const&	T	T | T const
T const&				T const&	T | T const	T | T const
T&&					T&&	T const&&
T const&&						T const&&

For most of the entries, the result of the conditional operator is the same regardless of whether `T` is a scalar type or a class type. For all of the marked entries (whether yellow or orange), the current behavior is that scalar types produce `T` and class types produce `T const`.

These themselves can be divided into two groups:

— the entries marked yellow have one operand that starts out as a const prvalue (i.e. `T const`).

— the entries marked orange have neither operand operand as a const prvalue.

Now, for those entries marked orange (including the two cases examined in this paper so far in the first row), the language is materializing a const prvalue itself. This just seems erroneous - the language shouldn't be doing this.

For those entries marked yellow, there was *already* a const prvalue. Here the question is different. Did the user really intend to create a const prvalue? If they did, maybe we should respect that intention. It's not entirely without merit to do so - it used to be a recommendation for functions to return `T const` so that `f() = x;` would be ill-formed. Although that recommendation went out of style with the adoption of move semantics (since doing so pessimizes assignment from `f()`) and with the adoption of ref-qualifiers for assignment (although these still don't seem to be very widely used). These cases also strike me as less important overall, simply because const prvalues don't come up very often (and will come up even less if we fix the oranges).

§ 3 Proposal

There are two potential proposals here: the *weak* proposal and the *strong* proposal.

The *weak* proposal: if both operands have the same underlying type (excluding value category and const), then the result of the conditional operator should only be a const prvalue if at least one of the operands is a const prvalue. Otherwise, it should be a non-const prvalue. That is, we change the orange entries below to be `T` for both scalar and class types:

?:	T	T const	T&	T const&	T&&	T const&&
T	T	T | T const	T	T	T	T

?:	T	T const	T&	T const&	T&&	T const&&
T const		T | T const	T | T const	T | T const	T | T const	T | T const
T&			T&	T const&	T	T
T const&				T const&	T	T
T&&					T&&	T const&&
T const&&						T const&&

The *strong* proposal: if both operands have the same underlying type (excluding value category and const), then the result of the conditional operator should never be a const prvalue (i.e. do as the `int`s do). That is, we change all of the orange and yellow entries to just be `T` for both scalar and class types:

?:	T	T const	T&	T const&	T&&	T const&&
T	T	T	T	T	T	T
T const		T	T	T	T	T
T&			T&	T const&	T	T
T const&				T const&	T	T
T&&					T&&	T const&&
T const&&						T const&&

The weak proposal is sufficient to address the pessimizing assignment issue and the const wrapping issue.

Notably, one odd quirk of the strong proposal is that the type of `cond<T, T>` is `T` for all types, value categories, and const. Except one: `T const`. In the strong proposal, `cond<T const, T const>` becomes `T`. Now, it technically is already just `T` for scalar types - but it's not possible to even have a const prvalue of scalar type, so this doesn't really matter. So we could alter the strong proposal to say that `cond<T, U>` is only ever a const prvalue in the specific case of `cond<T const, T const>` - otherwise all prvalues are non-const. This would still technically give different answers between scalar and class types, but not in a meaningfully observed way. This last version would produce this outcome (preserving the status quo for specifically `cond<T const, T const>`):

?:	T	T const	T&	T const&	T&&	T const&&
T	T	T	T	T	T	T
T const		T | T const	T	T	T	T
T&			T&	T const&	T	T
T const&				T const&	T	T
T&&					T&&	T const&&
T const&&						T const&&

§ 4 References

[P2278R4] Barry Revzin. 2022-06-17. cbegin should always return a constant iterator.
https://wg21.link/p2278r4